NLOpt

Natural-Language Optimization

A Controlled Natural Language DSL for Compiler Optimizations

Compiler Design Project

Presenters: Piyush Ajit Chavan -24BCE2330
Bhudev Singh Bhayal -24BCE2937
ARSHIA GOEL -24BCE2541

Traditional Compiler Optimizations Require Deep Specialized Expertise

NLOpt

Implementing compiler optimizations usually requires developers to understand complex intermediate representation (IR) semantics, control-flow properties, and compiler APIs.

- **Substantial Boilerplate Overhead:** Expressing a simple rule like replacing an addition by zero with the other operand currently requires substantial C++ boilerplate.
- **Manual Translation & Safety Constraints:** The current workflow forces developers to manually translate optimization intent into compiler-specific code while handling intricate safety constraints.
- **Implementation Gap:** This creates a significant gap between knowing what transformation is desirable and actually implementing it in the compiler backend.

NLOpt Bridges Human Optimization Intent with Formal Compiler Rules

NLOpt

NLOpt is a controlled-natural-language domain-specific language (DSL). It restricts vocabulary and sentence structure to remain unambiguous while remaining human-readable.

Input Phase

Human Specification

"For an addition with zero, replace the operation with the other operand."

Expresses high-level optimization logic using restricted natural language constructs.

Processing Pipeline

Transformation Flow

The controlled English statement undergoes a structured formal compilation pipeline:

Step 1
Abstract Syntax Tree (AST)

Step 2
Semantic Validation

Step 3
Internal Rule Representation

Output Phase

Deterministic Result

Generates an executable compiler transformation directly.

```
add(x, 0) -> x
```

Correctness Guaranteed

Operates without relying on unrestricted, opaque large language models.

NLOpt Directly Maps to Standard Compiler Construction Phases

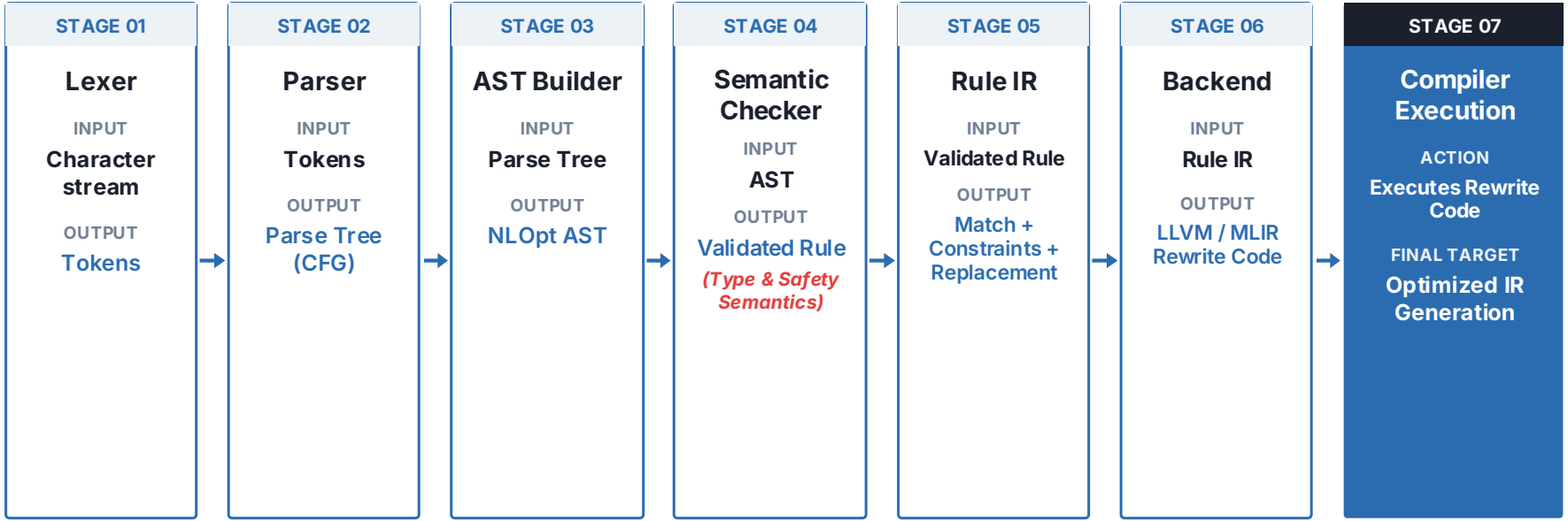
NLOpt

NLOpt is fundamentally a deterministic compiler construction project that utilizes standard phases:

- **Lexical Analysis:** Flex tokenizes controlled-natural-language keywords.
- **Syntax Analysis:** Bison checks if the sentence follows the predefined NLOpt grammar.
- **AST Construction:** Represents the English sentence structurally.
- **Semantic Analysis:** Checks transformation safety and meaningfulness.
- **Intermediate Representation:** Converts the AST into a formal pattern-rewrite representation.
- **Code Generation:** Emits LLVM or MLIR-oriented rewrite code.

The NLOpt Architecture Translates Text to Optimized Compiler Passes

The central pipeline transforms source specifications into optimized passes through a strict sequence:



NLOpt Compiler Pipeline Architecture Overview

NLOpt

Translating "Addition by Zero" into an LLVM Transformation

NLOpt

Human Specification

"For an addition with zero, replace the operation with the other operand."

NLOpt AST Representation

Operation: ADD

Condition: `operand2 == 0`

Replacement: `operand1`

IR Pattern Rule

`match: add(x, 0) | replace: x`

LLVM IR Transformation

// Before NLOpt

```
%t1 = add i32 %x, 0
%t2 = mul i32 %t1, 4
```

↓ Optimization Pass ↓

// After NLOpt

```
%t2 = mul i32 %x, 4
```

Structuring Natural Language into Machine-Processable Instructions

NLOpt

After parsing, the NLOpt sentence converts into a structured rule object.

AST Node Representation

Identifies core syntactical components from parsed input:

- **Operation:** Identifies the specific operation (e.g., ADD).
- **Operands:** Specifies input variables and terms.
- **Constraints:** Defines limitations (e.g., constant value).

Rule IR Structure

`match_operation:`
Identifies the IR operation to search for.

`operands:`
Defines the operand pattern.

`replacement:`
Defines the resulting IR expression.

`safety:`
Records whether the transformation is unconditional or condition-dependent.

Semantic Validation Prevents Unsafe Compiler Transformations

NLOpt

A syntactically valid rule must also preserve program semantics to be safe.

Hazard Example: The rule `x / x -> 1` is grammatically valid but semantically unsafe if x is zero. It must be rejected unless a non-zero condition is available.

Required Semantic Safety Validation Checks

Division by Zero

The rule `x / x -> 1` is grammatically valid but semantically unsafe if x is zero. It must be rejected unless a non-zero condition is available.

Type Checks

Integer-only rules cannot be applied to unsupported floating-point semantics.

Side Effects

Expressions performing observable I/O or memory effects cannot be safely removed.

Initial Minimum Viable Project Targets Fundamental Optimizations

NLOpt

The semester MVP focuses on core algebraic and peephole optimizations to prove the end-to-end pipeline:

OPTIMIZATION PHASE	TRANSFORMATION	PIPELINE OBJECTIVE / PURPOSE
Constant Folding	`3 + 4 → 7`	Tests semantic evaluation
Algebraic Simplification	`x + 0 → x`	Simple pattern rewrite
Multiplicative Identity	`x * 1 → x`	Demonstrates operand constraints
Multiplication by Zero	`x * 0 → 0`	Requires side-effect checking
Strength Reduction	`x * 2^n → x << n`	Selected multiplication to shift operations (target-aware)

Building the NLOpt Prototype with Industry-Standard Tools

NLOpt

Technology Stack

- **C++:** Core compiler framework implementation.
- **Flex:** Lexical analyzer to tokenize optimization keywords.
- **Bison:** Parser generator implementing the NLOpt CFG.
- **LLVM / MLIR:** Backend infrastructure generating intermediate representation (IR).

Expected Result & Workflow

Working Prototype Compiler Demo:

1. Write a controlled English optimization statement.
2. Execute the NLOpt compiler driver.
3. Inspect generated LLVM optimization pass code.
4. Apply transformations to a test C/C++ program.

Primary Objective Goal: **Achieve 100% parsing accuracy on the defined grammar test suite.**

Future Work Expands Optimization Scope and Backend Capabilities

NLOpt

Optimization

Expanded Rules

Advanced Analysis

Extending rules beyond the initial MVP scope to handle global transformations.

- Loop-invariant code motion
- Common subexpression elimination
- Complex data-flow analysis

Infrastructure

Advanced Backends

IR Integration

Incorporating state-of-the-art intermediate representations and rewrite engines.

- Full MLIR integration
- Multi-Level IR frameworks
- E-graph & equality saturation backend

Correctness

Formal Verification

Semantic Safety

Ensuring mathematical and semantic equivalence across optimizations.

- Alive2-style formal verification
- Translation validation pipelines
- Rigorous correctness proofs

Usability

LLM Frontend

Natural Grammar

Adding an intelligent natural language translation layer to the compiler pipeline.

- Optional LLM translation frontend
- Converts unrestricted English
- Strict NLOpt controlled grammar mapping